

Bases de données documentaires et distribuées
Cours NFE04
Système d'indexation : Solr

Auteurs : Raphaël Fournier-S'niehotta, Philippe Rigaux, Nicolas Travers
prenom.nom@cnam.fr

Département d'informatique
Conservatoire National des Arts & Métiers, Paris, France

Plan de la présentation

- 1 Installation, mise en route
- 2 Construisons notre moteur de recherche
- 3 Exploitation d'un index

Introduction

Systèmes d'indexation :

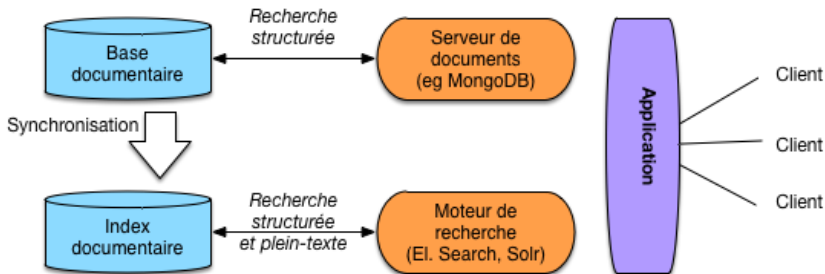
- Solr
 - Open Source
 - Fondation Apache
- Sphinx
- ElasticSearch

- ♠ Dans ce qui suit nous allons parler des documents Solr pour désigner la liste des champs transmise à l'index, et les distinguer des documents applicatifs de notre base de données documentaire
- ♠ Solr peut être vu comme une base de données spécialisée dans la recherche d'information : on y insère des documents (Solr), conformes à un schéma, et on peut rechercher des documents (Solr).

À quoi sert Solr ?

- Solr est une application spécialisée dans la recherche
- Repose sur l'index Lucene (Apache, auparavant projet distinct)
- Efficacité très élevée pour la recherche
- Moins pour les autres tâches de base de données (stockage, mise à jour fréquentes)
- Souvent c'est donc un **complément** d'un serveur de BDD (relationnelle ou documentaire)
- Spécialisé dans les recherches non structurées
- ex. : recherche par mot-clef dans un site Web
 - Documents gérés par MySQL
 - Extraction des textes et construction de l'index avec Solr
 - Le champ "Recherche" interroge Solr, qui répond

Architecture d'une application avec moteur de recherche



Installation

Installation très simple

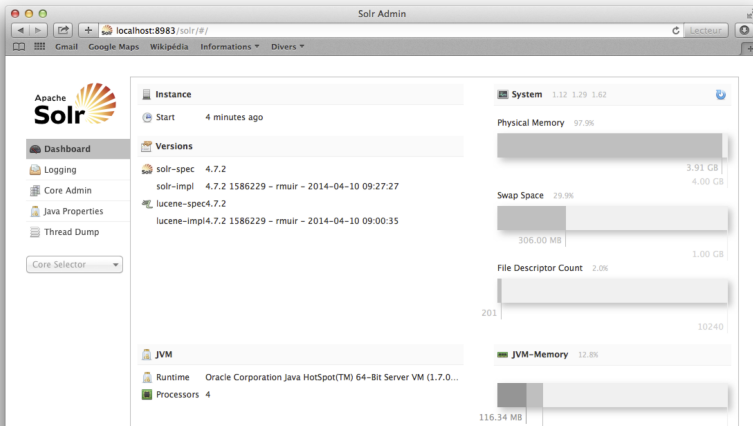
- pré requis : Java 1.7 (au moins)
- télécharger l'archive de la dernière version stable (4.10.2) :
`http://lucene.apache.org/solr/downloads.html`
- la décompresser quelque part ("soldir") :

```
tar -xvzf solr-4.10.2.tgz ~/soldir
```

Démarrage

- aller dans `/soldir/example` :
`cd ~/soldir/example`
- taper la commande suivante :
`java -jar start.jar`
- laisser ce terminal ouvert
- lancer un navigateur et aller à l'adresse :
`http://localhost:8983/solr`

Admin



Insertion de documents

- Solr fournit des services REST (HTTP) pour recevoir/envoyer des instructions
- codées en JSON ou XML
- insérer de documents
- effectuer des requêtes
- accessible à l'URL `http://localhost:8983/solr/update`

Exemple pour une indexation :

- attente de document de type "Content-type :application/xml" (binaire)
- on utilise curl :
`curl http://localhost:8983/solr/update -data-binary @item.xml -H 'Content-type :application/xml'`

Indexation en pratique

- dans le dossier “soldir/example/exampledocs/”, il y a par exemple :
solr.xml
- on peut utiliser curl, mais aussi deux utilitaires fournis par Solr :
 - post.jar :
java -jar post.jar solr.xml
 - post.sh :
./post.sh *.xml
- sélectionner ensuite le core “Collection1” dans la barre de navigation

Consultation de la Collection1

The screenshot shows the Solr Admin web interface in a browser window. The address bar indicates the URL is `localhost:8983/solr/#/collection1`. The interface is divided into several sections:

- Left Sidebar:** Contains the Apache Solr logo and a navigation menu with links to Dashboard, Logging, Core Admin, Java Properties, Thread Dump, and a dropdown menu for 'collection1'. Below this is a list of links: Overview (selected), Analysis, Dataimport, Documents, Files, Ping, Plugins / Stats, Query, and Replication.
- Statistics Section:** Displays various metrics for the collection:
 - Last Modified: 11 minutes ago
 - Num Docs: 32
 - Max Doc: 38
 - Heap Memory: 889
 - Usage:
 - Deleted Docs: 6
 - Version: 14
 - Segment Count: 2It also shows 'Optimized' and 'Current' status with red 'X' icons and an 'optimize now' button.
- Instance Section:** Lists configuration details:
 - CWD: `/Users/philippe/solr-4.7.2/example`
 - Instance: `/Users/philippe/solr-4.7.2/example/solr/collection1`
 - Data: `/Users/philippe/solr-4.7.2/example/solr/collection1/data`
 - Index: `/Users/philippe/solr-4.7.2/example/solr/collection1/data/index`
 - Impl: `org.apache.solr.core.NRTCachingDirectoryFactory`
- Replication (Master) Table:** A table showing the status of the master nodes.

	Version	Gen	Size
Master (Searching)	1412609270579	5	38.09 KB
Master (Replicable)	1412609313976	6	-
- Healthcheck Section:** Displays the message: 'Ping request handler is not configured with a healthcheck file.'
- Admin Extra:** A section for additional administrative links.
- Footer:** Contains links to Documentation, Issue Tracker, IRC Channel, Community forum, and Solr Query Syntax.

Contenus des documents indexés

```
<add>
  <doc>
    <field name="id">SOLR1000</field>
    <field name="name">Solr, the Enterprise Search Server</field>
    <field name="manu">Apache Software Foundation</field>
    <field name="cat">software</field>
    <field name="cat">search</field>
    <field name="features">Advanced Full-Text Search Capabilities using Lucene</fi
    <field name="features">Optimized for High Volume Web Traffic</field>
    <field name="price">0</field>
    <field name="popularity">10</field>
    <field name="inStock">>true</field>
    <field name="incubationdate_dt">2006-01-17T00:00:00.000Z</field>
  </doc>
</add>
```

- Le document est constitué de **champs (fields)**,
- le nom du champ est indiqué par l'attribut **name**.
- il faut qu'ils aient été déclarés au préalable dans le **schéma** de l'index.
 - **price** est déclaré comme numérique
 - **features** comme chaîne de caractères

Interroger un index avec Solr

- Solr dispose d'une interface REST pour **rechercher** des documents :

```
http://localhost:8983/solr/select?q=video
```

- dans le navigateur, ou avec curl :

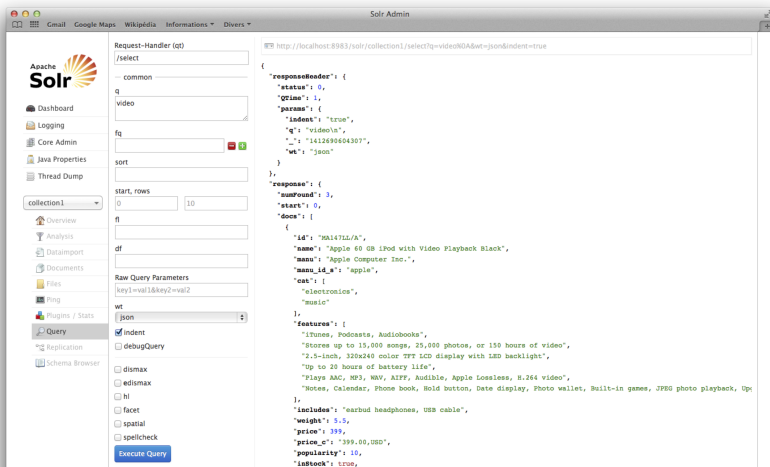
```
\curl http://localhost:8983/solr/select?q=video
```

- La réponse est formatée en XML. On peut aussi demander un retour en JSON :

```
http://localhost:8983/solr/select?q=video&wt=json&indent=yes
```

- interface d'administration intégrée

Fenêtre d'interrogation de Solr



The screenshot displays the Solr Admin web interface. On the left is a sidebar with navigation links: Dashboard, Logging, Core Admin, Java Properties, Thread Dump, Overview, Analysis, Dataimport, Documents, Files, Ping, Plugins / Stats, Query (selected), Replication, and Schema Browser. The main area is titled 'Request-Handler (qt)' and shows the path '/select'. Below this, there are input fields for 'q' (containing 'video'), 'fq', 'sort', 'start, rows' (0 to 10), and 'fl'. The 'Raw Query Parameters' section shows 'key1=val1&key2=val2'. The 'wt' dropdown is set to 'json', and the 'indent' checkbox is checked. Other checkboxes for 'debugQuery', 'dismax', 'edismax', 'hl', 'facet', 'spatial', and 'spellcheck' are present. A blue 'Execute Query' button is at the bottom. The right pane shows the JSON response for the query 'http://localhost:8983/solr/collection1/select?q=video&wt=json&indent=true'. The response is a JSON object with a 'responseHeader' and a 'response' containing a list of documents. The first document is an 'Apple 60 GB iPod with Video Playback Black' with various attributes like 'id', 'name', 'manu', 'manu_id_s', 'cat', 'features', 'includes', 'weight', 'price', 'price_c', 'popularity', and 'inStock'.

```
{
  "responseHeader": {
    "status": 0,
    "QTime": 1,
    "params": {
      "indent": "true",
      "q": "video",
      "wt": "json"
    }
  },
  "response": {
    "numFound": 3,
    "start": 0,
    "docs": [
      {
        "id": "M4147LL/A",
        "name": "Apple 60 GB iPod with Video Playback Black",
        "manu": "Apple Computer Inc.",
        "manu_id_s": "apple",
        "cat": [
          "electronics",
          "music"
        ],
        "features": [
          "iTunes, Podcasts, Audiobooks",
          "Stores up to 15,000 songs, 25,000 photos, or 150 hours of video",
          "2.5-inch, 320x240 color TFT LCD display with LED backlight",
          "Up to 20 hours of battery life",
          "Plays AAC, MP3, WAV, AIFF, Audible, Apple Lossless, H.264 video",
          "Notes, Calendar, Phone book, Hold button, Date display, Photo wallet, Built-in games, JPEG photo playback, Up"
        ],
        "includes": "earbud headphones, USB cable",
        "weight": 5.5,
        "price": 399,
        "price_c": "399.00,USD",
        "popularity": 10,
        "inStock": true,

```

Réponse de Solr

- En-tête (propriétés sur l'exécution, dont temps de réponse)
 - Liste des **documents Solr** considérés comme satisfaisant les critères
 - constitués des valeurs de champs insérés dans l'index
 - cela ne suffira cependant souvent pas, il faudra accéder au **document applicatif** correspondant
 - importance du champ **id** (clef d'accès base documentaire)
- ♠ Certains champs (**cat** et **features** par exemple) sont multivalués (ils sont représentés par des tableaux en JSON). L'index en a tenu compte

Illustration de la puissance de Solr

- recherche par mot-clef
- **projection** (SQL) avec le paramètre **fl** (**field list**) :

```
http://localhost:8983/solr/select?q=video&fl=id,name
```

- spécifier dans quel champ on cherche :

```
http://localhost:8983/solr/select?q=name:black
```

- pagination de résultats :

```
http://localhost:8983/solr/select?q=video&start=1&rows=10
```

Plan de la présentation

- 1 Installation, mise en route
- 2 Construisons notre moteur de recherche
- 3 Exploitation d'un index

Construction d'un moteur de recherche

- On va utiliser la base des films de MongoDB et l'indexer
- Dans Solr, un index s'appelle **core**
- Celui par défaut que l'on a utilisé s'appelait "collection1"
- On va en configurer un autre, "movies"
- Dans solr/example/solr :

```
cp -R collection1 movies  
cd movies
```

- éditer "core.properties" pour changer la propriété name à movies
- vider data/ :

```
rm -Rf data/*
```

- se placer dans le répertoire "conf"

Un document (exemple)

```
{  
  "_id": "movie:57",  
  "title": "Jackie Brown",  
  "year": "1997",  
  "genre": "crime",  
  "summary": "Jackie Brown, hôtesse de l'air, arrondit ses fins de mois ()",  
  "country": "USA",  
  "director": "Quentin Tarantino",  
  "actors": ["Robert De Niro", "Pam Grier", "Bridget Fonda", "Michael Keaton", "Sa  
}
```

Schéma de l'index

- Le schéma donne la liste de tous les champs d'un doc Solr
- Nombreuses options :
 - type (numérique, entier)
 - possibilité de calcul de la valeur du champ à partir d'un autre
 - traitements divers sur les valeurs du champ

Squelette

```
<?xml version="1.0" encoding="UTF-8" ?>

<schema name="example" version="1.5">
  <!-- Liste des champs de l'index -->
  <fields>
    <field name="_id" type="string" indexed="true" stored="true" required="true" />
    <field name="title" type="string" indexed="true" stored="true" required="true" />
    <field name="summary" type="text" indexed="true" stored="false" required="false" />
    <!-- A compléter -->

    <!-- Un champ dans lequel on concatène les autres pour une recherche "plein-texte" -->
    <field name="text" type="text" indexed="true" stored="false"
      multiValued="true" />
    <copyField source="summary" dest="text" />
    <copyField source="title" dest="text" />

    <!-- Un champ "technique" requis par Solr/Lucene -->
    <field name="_version_" type="long" indexed="true" stored="true" />
  </fields>

  <!-- La clé d'accès à un document dans l'index -->
  <uniqueKey>_id</uniqueKey>

  <!-- Configuration des types de champ -->
  <types>
    <fieldType name="string" class="solr.StrField" />
    <fieldType name="int" class="solr.IntField" />
    <fieldType name="long" class="solr.LongField" />
    <fieldType name="text" class="solr.TextField">
      <analyzer>
        <tokenizer class="solr.StandardTokenizerFactory" />
        <filter class="solr.LowerCaseFilterFactory" />
      </analyzer>
    </fieldType>
  </types>
</schema>
```

Squelette de l'index

- la liste des champs, dans l'élément "fields", complétée par l'indication du champ de recherche par défaut ;
- le champ qui identifie le document Solr, dans l'élément "uniqueKey" ;
- la liste des **types de champ**, dans l'élément "types".

Note : Pour des besoins internes, tout schéma doit contenir un champ "`_version_`" défini comme ci-dessus.

Définition des types et de la clé

- Chaque type utilisé dans le schéma d'un index doit apparaître dans un des éléments **fieldType** du fichier **schema.xml**
- Solr fournit tout un ensemble de types pré-définis qui suffisent pour les besoins courants
- Pour simplifier, disons qu'un type Solr correspond à une classe Java (du nom du type)
- On peut associer des options à un type
- Les options indiquent d'éventuels traitements à appliquer à chaque valeur du type avant son insertion dans l'index
- ex : type **text**
 - on lui définit un "analyseur" "StandardTokenizerFactory"
 - se charge de découper le texte en **tokens** pour une recherche plein-texte (détails plus tard)
 - retenir : cela permet d'indexer chacun des mots, et donc de faire des recherches sur toutes les combinaisons de mots
- L'élément **uniqueKey** permet de rechercher un document dans l'index par sa clé. Indispensable, ne serait-ce que pour savoir qu'un document est indexé
(le définir systématiquement semble une bonne pratique)

Définition des champs

```
<field name="_id" type="string" indexed="true" stored="true"
      required="true" multiValued="false" />
```

- Les attributs de l'élément XML caractérisent le champ
- Le **nom** et le **type** sont les informations de base
- Ensuite, divers attributs (souvent optionnels) :
 - **indexed** indique simplement que le champ peut être utilisé dans une recherche ;
 - **stored** indique que la **valeur** du champ est **stockée** dans l'index, et qu'il est donc possible de récupérer cette valeur comme résultat d'une recherche, **sans avoir besoin de retourner à la base principale** ; en d'autres termes, "stored" permet de traiter l'index **aussi** comme une base de données ;
 - **required** indique que le champ est obligatoire ;
 - enfin, **multiValued** vaut **true** pour les champs ayant plusieurs valeurs, soit, concrètement, un **tableau** en JSON ; c'est le cas par exemple pour le nom des acteurs.

Définition des champs

- Les champs **indexed** et **stored** sont très importants
- toutes les combinaisons de valeur sont possibles :
 - **indexed=true, stored=false** : on pourra interroger le champ, mais il faudra accéder au document principal dans la base documentaire si on veut sa valeur ;
 - **indexed=true, stored=true** : on pourra interroger le champ, **et** accéder à sa valeur dans l'index ;
 - **indexed=false, stored=true** : on ne peut pas interroger le champ, mais on peut récupérer sa valeur dans l'index ;
 - **indexed=false, stored=false** : n'a pas de sens à priori ; le seul intérêt est d'ignorer le champ s'il est fourni dans le document Solr.

Définition des champs (suite)

comment peut-on indexer un champ sans le stocker ?

- c'est notamment le cas pour les textes qui sont décomposés en **termes**
- chaque terme est indexé indépendamment
- très difficile pour l'index de reconstituer le texte
- d'où l'intérêt de conserver ce dernier dans son intégralité, à part

C'est une question de compromis :

- **stocker** une valeur prend plus d'espace que **l'indexer**
- Dans la situation la plus extrême, on dupliquerait la base documentaire en stockant chaque document **aussi** dans l'index
- un stockage plus important dégrade les performances

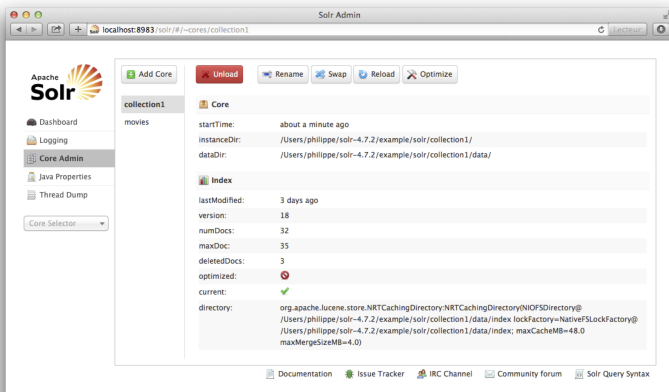
Y a-t-il une valeur par défaut pour ces options ?

- les valeurs par défaut de **indexed** et **stored** par exemple sont **héritées** du type du champ (par exemple **TextField**),
- pour le type, on ne sait pas toujours comment c'est défini.
- il est donc **préférable de toujours les mettre explicitement**

- Le squelette de schéma comprend également un champ *calculé*, le champ **text**.
- Les instructions **copyField** indiquent qu'au moment de l'insertion d'un document, on va "copier" certains champs dans celui-ci.
 - le type du champ **destination** correspond à un mode particulier d'indexation, éventuellement différent et complémentaire de celui du champ **origine** ; par exemple le contenu d'un titre est indexé comme une chaîne de caractères dans le champ **title**, et comme un texte "tokenisé" quand on le copie dans le champ **text** ;
 - si **toutes** les occurrences de chaînes de caractères sont placées dans un même champ, on obtient, en prenant ce champ pour cible, une recherche plein-texte globale, une fonctionnalité souvent utile.

Recharger le schéma

- Après tout changement de schéma, il faut **recharger** l'index.
- Pour recharger un index, à partir de l'interface d'administration, utilisez l'option **Reload** après avoir sélectionné le **core**



Rechargement

```
curl 'http://localhost:8983/solr/movies/update/json?commit=true' \\  
--data-binary @solr\_doc.json -H 'Content-type:application/json'
```

- Attention, si l'index existant ne correspond pas au nouveau schéma, le rechargement échouera.
- Avec Solr, il est (plus) difficile de faire évoluer un schéma (qu'avec BDD classique)

Reconstruction (destruction puis validation) :

```
curl http://localhost:8983/solr/movies/update \\  
--data '<delete><query>*:*/</query></delete>' -H 'Content-type:text/xml;  
curl http://localhost:8983/solr/movies/update \\  
--data '<commit/>' -H 'Content-type:text/xml; charset=utf-8'
```

Plan de la présentation

- 1 Installation, mise en route
- 2 Construisons notre moteur de recherche
- 3 Exploitation d'un index

Recherche de documents

- Possibilités très sophistiquées de Solr
- La manière d'effectuer une recherche varie en fonction de :
 - la syntaxe de la requête (de très structurée avec booléen sur des champs) à liste de mots-clefs
 - le **classement** du résultat
 - Solr renvoie les documents "pertinents" (pas seulement ceux qui correspondent exactement aux critères)
 - différence avec BDD (relationnelle)

Consulter l'index

- le paramètre qu'on utilise ici, c'est "q" (**query**)
- proposé par défaut dans l'interface comme "*.*" (**tous** les documents de l'index)
- **fq** : pour **filter query**, pour interroger non pas l'index entier mais un résultat pré-calculé et stocké en **cache**
- **sort**, pour trier le résultat
- **start** et **row**, les paramètres classiques de pagination du résultat
- **fl** pour **field list**, la liste des champs (stockés) à inclure dans le résultat
- **df**, le champ à interroger si non spécifié dans la requête (la valeur par défaut est indiqué dans la configuration et vaut en principe **text**, le champ dans lequel nous avons placé toutes nos chaînes de caractères) ;
- enfin, on trouve la liste des **query parsers** disponibles ;
un **query parser** correspond à une syntaxe d'interrogation particulière.

Exemple d'utilisation des paramètres

- avec **sort=title asc** : tri du résultat sur le titre, en ordre ascendant ;
- avec **fl=title, year**, restriction des champs dans les documents du résultat ;
- avec **start=10, rows=9**, récupération des documents classés entre les positions 10 à 19 ;
- avec **q=Alien**, vous devriez retrouver le document Solr correspondant au film **Alien** ;
- avec **q=Alien** mais **df=summary**, vous ne devriez rien trouver ;
- avec **q=Vertigo, df=text**, vous devriez retrouver le film Vertigo ;

Paramètres

- le document résultat ne montre que ceux qui ont été définis dans le schéma comme “stockés”
- les autres champs sont utiles pour la recherche, mais on ne peut pas récupérer leur valeur
- En revanche, il est possible d'obtenir des informations **calculées** par Solr, sous forme de (pseudo-)champ :
 - ex. : le **score**, qui évalue la pertinence d'un document pour une recherche
 - essayer avec **title, year, score** dans **fl** (et une recherche par mot-clef (ex. : **fin**))
 - on vérifie que l'on a un classement par score
 - essayer en rajoutant **[explain style=nl]**

Les requêtes

- Solr fournit plusieurs interpréteurs de requêtes
- chacun reconnaît des syntaxes légèrement différentes
- l'interpréteur par défaut, "DisMax", est le plus intuitif
- mais pas toujours le plus précis

Termes

- Notion de base : le **terme**
- c'est un mot au sens usuel
- ou une séquence de mots entre apostrophes

Interroger l'index "collection1" avec :

hard drive

Puis :

"hard drive"

- Première recherche : documents avec "hard", "drive" ou les deux
- Deuxième : seulement "hard drive" (côte à côte)

Termes (suite)

- Dans Solr, la recherche d'un terme s'effectue toujours sur un champ.
- La syntaxe complète pour associer le champ et le terme est :

```
champ:terme
```

- si non précisé, c'est le champ par défaut qui est utilisé
- pratique courante : concaténer toutes les chaînes de caractères en un champ "text" général, défini par défaut
- Nos requêtes deviennent :

```
text:hard text:drive
```

- et

```
text:"hard drive"
```

Termes (suite)

- Les valeurs des termes (dans la requête) et le texte indexé sont tous deux soumis à des transformations spécifiées dans le schéma.
- Une transformation simple est de tout transcrire en minuscules.

```
text: "Hard Drive"
```

- Les transformations appliquées à la requête ET au texte indexé doivent être cohérentes : si les termes sont transformés en majuscules, et le texte indexé en minuscules, on n'aura jamais de résultat !

Termes (suite)

On peut spécifier des termes (pas des séquences) incomplets

- le '?' indique un caractère inconnu
 - "opti?al" désigne "optimal", "optical", etc.
- le '*' indique n'importe quelle séquence de caractères
 - "opti*" pour toute chaîne commençant par "opti"

Approximations avec "~" :

- Rechercher "optimal" et "optimal~"
- 0 et 1 résultat ("optical")
- Proximité des termes par une distance d'édition :
(nb opérations pour passer de "optimal" à "optical")

Intervalles :

- [] bornes comprises
- { } bornes exclues

```
%price:[100 TO 200]
```

Requêtes Booléennes

- Les critères peuvent être combinés avec des opérateurs Booléens :
AND, **OR** et **NOT**
- Attention : majuscules

```
%price:[100 TO 300] OR popularity:5  
%price:[100 TO 300] AND NOT popularity:5  
%popularity:6 AND features:matrix
```

- Par défaut, c'est un **OR** qui est appliqué
- Recherche sur plusieurs critères ramène l'union des résultats sur chaque critère pris individuellement
- La requête suivante recherche les produits Dell **ou** dont la popularité est égale à 6 :

```
%popularity:6 manu:Dell
```

Opérateur +, classement

- préfixe d'un nom de champ, il indique que la valeur du champ **doit** être égale au terme
- il existe également un opérateur **-**, équivalent au **NOT**
- recherche des documents dont la popularité est 6 (obligatoire) et qui peuvent être produits par Dell ou un autre constructeur

```
%+popularity:6 manu:Dell
```

- différence avec ce qui précède : le **classement** du moteur
- illustre la différence entre recherche d'information et interrogation de bases de données

Interpréter un classement est parfois délicat :

```
%+popularity:6 cat:electronics
```

```
%+popularity:6 -manu:Dell
```
